

EXPERIMENT - 43

1. Problem Statement

Write a menu-driven C program to perform the following operations using separate functions:

- Find the longest word in a sentence.
- Check if one string is a rotation of another.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu and read choice.

Step 3: Using a switch statement on choice:

If choice is 1, read a sentence, call findLongestWord() and display the word it fills in.

If choice is 2, read two strings, call isRotation() and display whether the second is a rotation of the first.

If choice is 3, display "Exiting the program."

Step 4: Repeat from Step 2 while choice is not equal to 3.

Step 5: STOP

Function findLongestWord()

Step 1: Copy sentence into temp and set longestWord to an empty string.

Step 2: Get the first word of temp using strtok().

Step 3: Repeat while the word is not NULL:

If strlen(word) is greater than strlen(longestWord), copy word into longestWord.

Get the next word using strtok() with NULL.

Step 4: Return to the calling function.

Function isRotation()

Step 1: If strlen(s1) is not equal to strlen(s2), return 0.

Step 2: Copy s1 into joined using strcpy(), then append s1 to it again using strcat().

Step 3: If strstr(joined, s2) is not NULL, return 1.

Step 4: Return 0.

3. Test Case Table

#	Operation	Input	Expected Output	Actual Output	Result
1	1. Longest word	C programming is a powerful language	programming, at 11 letters	programming, the other words are all shorter	PASS

#	Operation	Input	Expected Output	Actual Output	Result
2	1. Longest word	I love learning data structures	structures, which beats learning by two	structures, not learning	PASS
3	1. Longest word	Practice makes a person perfect.	Practice, since perfect is only seven letters once the full stop is dropped	Practice, so the dot was not counted	PASS
4	2. Rotation	abcde and cdeab	It is a rotation	Said it is a rotation	PASS
5	2. Rotation	abcde and abced	Not a rotation, only the last two letters are swapped	Said it is not	PASS
6	2. Rotation	waterbottle and erbottlewat	It is a rotation	Correct on a longer pair too	PASS
7	2. Rotation	hello and helloo	Not a rotation, the lengths differ	Said it is not, caught by the length test	PASS
8	2. Rotation	listen and listen	It is a rotation, a rotation by zero	Said it is a rotation	PASS
9	3. Exit	choice = 3	Exiting the program, and the loop ends	Loop ended	PASS

4. Source Code

strops43.c

```
/* Experiment 43. One function for each of the two operations.
   Each option reads its own input, because option 1 wants a whole
   sentence and option 2 wants two single words, so there was nothing
   sensible I could read once at the start and share between them.
   The question lists two operations and no exit, but a do while menu
   has nothing to stop it, so option 3 is mine. Without it the menu
   just keeps coming back. */

#include <stdio.h>
#include <string.h>

void findLongestWord(const char sentence[], char longestWord[]);
int isRotation(const char s1[], const char s2[]);

int main(void)
{
    char sentence[200], longestWord[50];
    char str1[100], str2[100];
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Find the longest word in a sentence\n");
        printf("2. Check whether one string is a rotation of another\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                /* A sentence has spaces in it, so %s is no good here,
                   it would stop at the first one. */
                printf("Enter a sentence: ");
                scanf(" %[^\\n]", sentence);
                findLongestWord(sentence, longestWord);
                printf("Longest word: %s\\n", longestWord);
                break;

            case 2:
                /* These two are single words, so %s is fine and it
                   also throws away any stray spaces for me. */
                printf("Enter the first string: ");
                scanf(" %s", str1);
                printf("Enter the second string: ");
                scanf(" %s", str2);
                if (isRotation(str1, str2))
                    printf("The second string is a rotation of the first\\n");
                else
                    printf("The second string is not a rotation of the first\\n");
                break;

            case 3:
                printf("Exiting the program.\\n");
                break;
        }
    } while (choice != 3);

    return 0;
}

/* strtok gives back one word per call and NULL once the sentence runs
   out. The string goes in on the first call and NULL on every call
   after it, which is how strtok keeps track of where it had stopped.
   temp is not me being careful, it is the only thing that works.
   strtok does not read a string, it cuts it up, writing '\\0' wherever
   a separator sits, and the sentence arrives as const, so handing it
   over directly would not even compile.
```

The full stop and the comma sit in the separator list next to the space. Without them perfect. would be measured as eight letters and would have beaten Practice, which is the real answer. */

```
void findLongestWord(const char sentence[], char longestWord[])
{
    char temp[200];
    char *word;

    strcpy(temp, sentence);

    /* Emptying longestWord first is what lets the comparison below be
       one plain line. strlen of an empty string is 0, so the very
       first word beats it automatically and there is no need for a
       separate case to get things started. Leaving this out gave me
       rubbish, because the array still held whatever was in memory. */
    longestWord[0] = '\0';

    word = strtok(temp, " \\t\\n.,!?:;");
    while (word != NULL) {
        if (strlen(word) > strlen(longestWord))
            strcpy(longestWord, word);

        word = strtok(NULL, " \\t\\n.,!?:;");
    }
}
```

/* My first plan was to shift the string along by one letter, compare, shift again, compare, and keep at it until every position had been tried. Then I noticed that a rotation only moves letters from the front round to the back, so every rotation of abcde is already hiding inside abcdeabcde. Join the string to itself once and one strstr tests the whole lot.

The length check is not just a shortcut, it is what makes the method correct. ab sits quite happily inside abcabc, so without it abc and ab would have been called a rotation of each other, and a rotation never loses letters. */

```
int isRotation(const char s1[], const char s2[])
{
    char joined[200];

    if (strlen(s1) != strlen(s2))
        return 0;

    strcpy(joined, s1);
    strcat(joined, s1);

    if (strstr(joined, s2) != NULL)
        return 1;

    return 0;
}
```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra strops43.c -o strops43`. Those two flags show up even the small warnings, and it built clean.

The Longest Word in Three Different Sentences

```
adilshakil — -zsh — 122x30
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra strops43.c -o strops43
adilshakil@ADILs-MacBook-Air MyAssignment % ./strops43

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 1
Enter a sentence: C programming is a powerful language
Longest word: programming

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 1
Enter a sentence: I love learning data structures
Longest word: structures

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 1
Enter a sentence: Practice makes a person perfect.
Longest word: Practice
█
```

Rotations That Work and One That Does Not

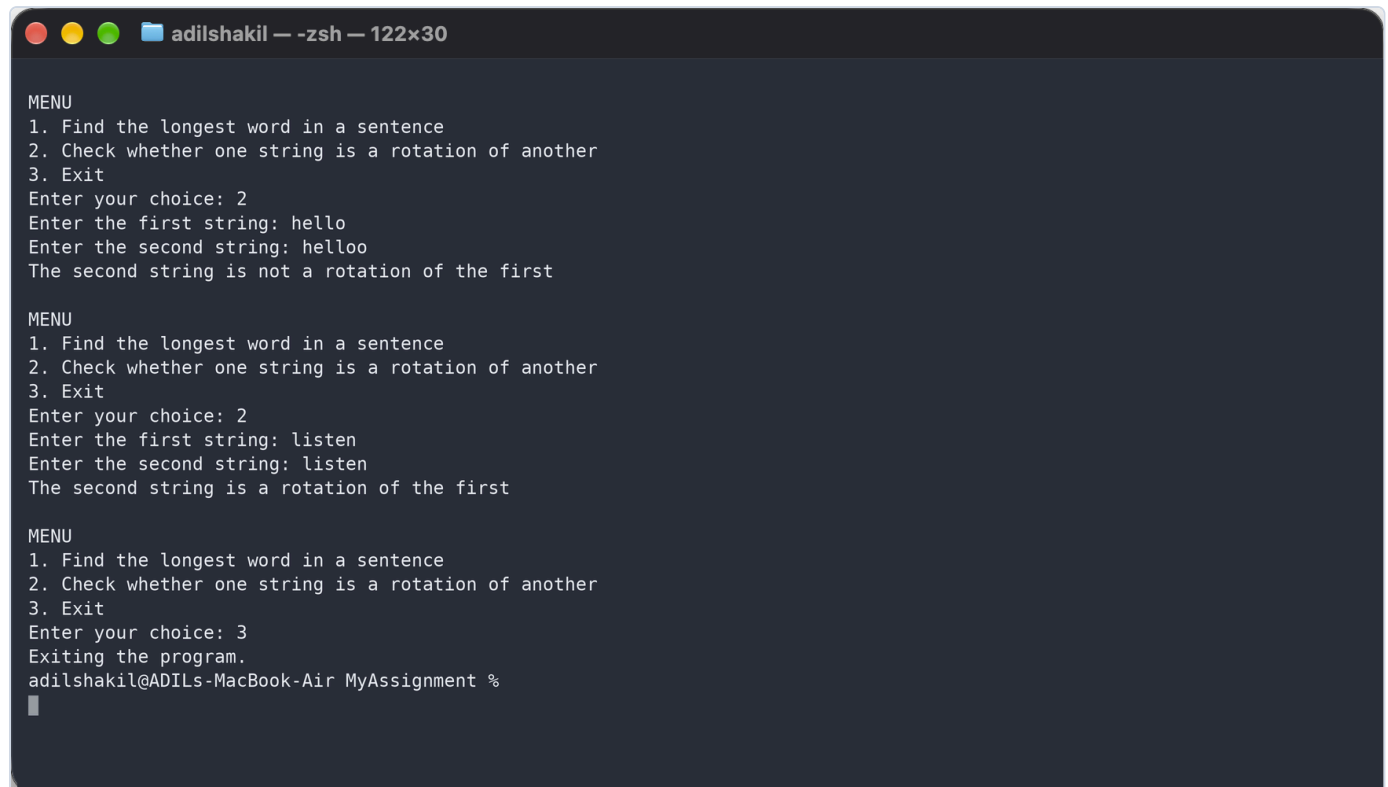
```
adilshakil — -zsh — 122x30

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 2
Enter the first string: abcde
Enter the second string: cdeab
The second string is a rotation of the first

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 2
Enter the first string: abcde
Enter the second string: abced
The second string is not a rotation of the first

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 2
Enter the first string: waterbottle
Enter the second string: erbottlewat
The second string is a rotation of the first
█
```

A Length Mismatch, a String Against Itself, and Exit



```
adilshakil — -zsh — 122x30

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 2
Enter the first string: hello
Enter the second string: helloo
The second string is not a rotation of the first

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 2
Enter the first string: listen
Enter the second string: listen
The second string is a rotation of the first

MENU
1. Find the longest word in a sentence
2. Check whether one string is a rotation of another
3. Exit
Enter your choice: 3
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All nine cases came out right.

Option 2 is the one I enjoyed. My first plan was the obvious one. Shift the first string along by a letter, compare it with the second, shift again, compare again, and carry on until every position has been tried. It works, but it is a lot of shuffling for something that ought to be simple. Then I noticed what a rotation actually does: it takes letters off the front and puts them on the back, and nothing else. So every rotation of abcde is already sitting inside abcdeabcde. Join the string to itself once and one strstr tests all of them together.

The length check at the top of that function is not a shortcut, it is what keeps the method honest. I nearly left it out, and then thought about what could sneak through. ab sits quite happily inside abcabc, so abc and ab would have been declared a rotation of each other. They are not, because a rotation shifts letters around and never drops any.

Case 8 is there on purpose. listen against listen comes back as a rotation, which stopped me for a second, but it is right. Rotating by zero places leaves a string exactly as it was, and a string does contain itself.

Option 1 would not compile at all to begin with. strtok does not just read through a string, it cuts it up, writing a '\0' wherever it finds a separator, and the sentence arrives as const char sentence[]. Copying it into temp and letting strtok take the copy apart fixed it, and it has the side benefit that the sentence the user typed is never damaged.

The other thing in option 1 is the line that empties longestWord before the loop. I had left it out, thinking an array starts off empty, and got rubbish back. It does not start off empty, it holds whatever happened to be in that bit of memory. Once it is emptied, strlen of it is 0, so the first word beats it on its own and the comparison inside the loop can stay a single plain line.

I also put the full stop and the comma into the separator list next to the space. In Practice makes a person perfect. the word perfect has a dot stuck to it, and without them the program would have measured it as eight letters and called it the winner. Practice is the real answer at eight, and perfect is only seven.

Last thing. The question lists two operations and no exit, but a menu built on a do while loop has nothing to stop it, so option 3 is mine. Without it the menu simply keeps coming back.